# DeepStream optical character recognition (OCR)

**DeepStream optical character recognition (OCR)**
1. The NVIDIA official application (for reference only)
2. Case goal and processing chain
3. Model version and applicable scope
4. Course script entry points and output
5. Running a local video
   5.1 Run
   5.2 Replace the video
   5.3 Output location
6. Real-time recognition with a USB camera
   6.1 Run
   6.2 Camera pre-check
7. Real-time recognition with a CSI camera
   7.1 Run directly
   7.2 CSI no-picture check
8. Notes on the current implementation
9. Stopping and troubleshooting
10. Quick reference of the key file paths
11. References

> Applicable environment: JetPack 7.2.1, Ubuntu 24.04, DeepStream 9.1, CUDA 13.2, Jetson Orin NX.

> The factory system has already completed the model download, the TensorRT engine build, the application compilation and the deployment of the course entry. For normal study, start directly from section 3; only refer to section 1 to restore the environment when the program, the model or the engine is missing.

## 1. The NVIDIA official application (for reference only)

This case uses the NVIDIA DeepStream official `deepstream-nvocdr-app`, whose source code is located at:

```
~/deepstream_tao_apps/apps/tao_others/deepstream-nvocdr-app
```

Official documentation: https://github.com/NVIDIA/DeepStream/tree/main/src/apps/tao_others/deepstream-nvocdr-app

nvocdr uses the **customlib architecture** (not the nvinfer plugin), so the engine must be **generated manually in advance**. When you restore the environment for the first time, execute the following in order:

```
# 1. Download the OCDNet / OCRNet models (the non ViT version)
mkdir -p ~/deepstream_tao_apps/models/nvocdr
cd ~/deepstream_tao_apps/models/nvocdr

curl -L -o ocdnet_dcn_resnet18.onnx \
```

```
  'https://api.ngc.nvidia.com/v2/models/org/nvidia/team/tao/ocdnet/deployable_v1.0/files?
redirect=true&path=dcn_resnet18.onnx'

curl -L -o ocrnet_resnet50.onnx \
  'https://api.ngc.nvidia.com/v2/models/org/nvidia/team/tao/ocrnet/deployable_v1.0.1/files?
redirect=true&path=ocrnet_resnet50.onnx'

curl -L -o character_list \
  'https://api.ngc.nvidia.com/v2/models/org/nvidia/team/tao/ocrnet/deployable_v1.0/files?
redirect=true&path=character_list'

# 2. Generate the engine with the trtexec of the system TRT 11
/usr/bin/trtexec --onnx=ocdnet_dcn_resnet18.onnx \
  --minShapes=input:1x3x736x1280 \
  --optShapes=input:1x3x736x1280 \
  --maxShapes=input:4x3x736x1280 \
  --memPoolSize=workspace:4096 \
  --saveEngine=ocdnet.fp16.engine

/usr/bin/trtexec --onnx=ocrnet_resnet50.onnx \
  --minShapes=input:1x1x32x100 \
  --optShapes=input:32x1x32x100 \
  --maxShapes=input:32x1x32x100 \
  --memPoolSize=workspace:2048 \
  --saveEngine=ocrnet_resnet50.fp16.engine
```

**Note**: TRT 11 has no `--fp16` parameter, so writing only `--memPoolSize` is enough.

# 2. Case goal and processing chain

This case is based on the NVIDIA official nvocdr: it detects the text regions in the frame, recognizes the characters in them, and outputs the recognized text content in the results.

The complete processing chain is as follows:

```
Input video or camera
        ↓
OCDNet: text region detection (segmentation mask)
        ↓
Crop + batch resize
        ↓
OCRNet: character recognition for each region
        ↓
Character decoding (character_list)
        ↓
OSD overlays the text labels
```

Unlike a pure nvinfer solution, OCDNet outputs a segmentation mask, and OCRNet has to crop every detection box and then run the inference in batches. NVIDIA provides nvocdr precisely because integrating OCR with a pure nvinfer is extremely complex. **nvocdr is the officially recommended solution**.

# 3. Model version and applicable scope

| Parameter | Model used | Applicable content |
| --- | --- | --- |
| nvocdr default | `ocdnet_dcn_resnet18 + ocrnet_resnet50` | Digits, lowercase English letters |

The current character_list contains only **36 characters (0-9, a-z)**, and it **does not support Chinese, uppercase letters or symbols**. If you need to recognize Chinese license plates or uppercase English, you have to replace the dictionary file or the model with the corresponding one.

# 4. Course script entry points and output

The current unified startup script is located in `~/deepstream_tao_demo/`:

```
cd ~/deepstream_tao_demo
```

| Input source | Startup command |
| --- | --- |
| Local video | `./run_ocr.sh` |
| USB camera | `./run_ocr_usb.sh` |
| CSI camera | `./run_ocr_csi.sh` |

All three entry points display a real-time window on the local Jetson desktop or the VNC desktop. The USB/CSI entry points only display in real time; the local video entry point accepts a custom video path.

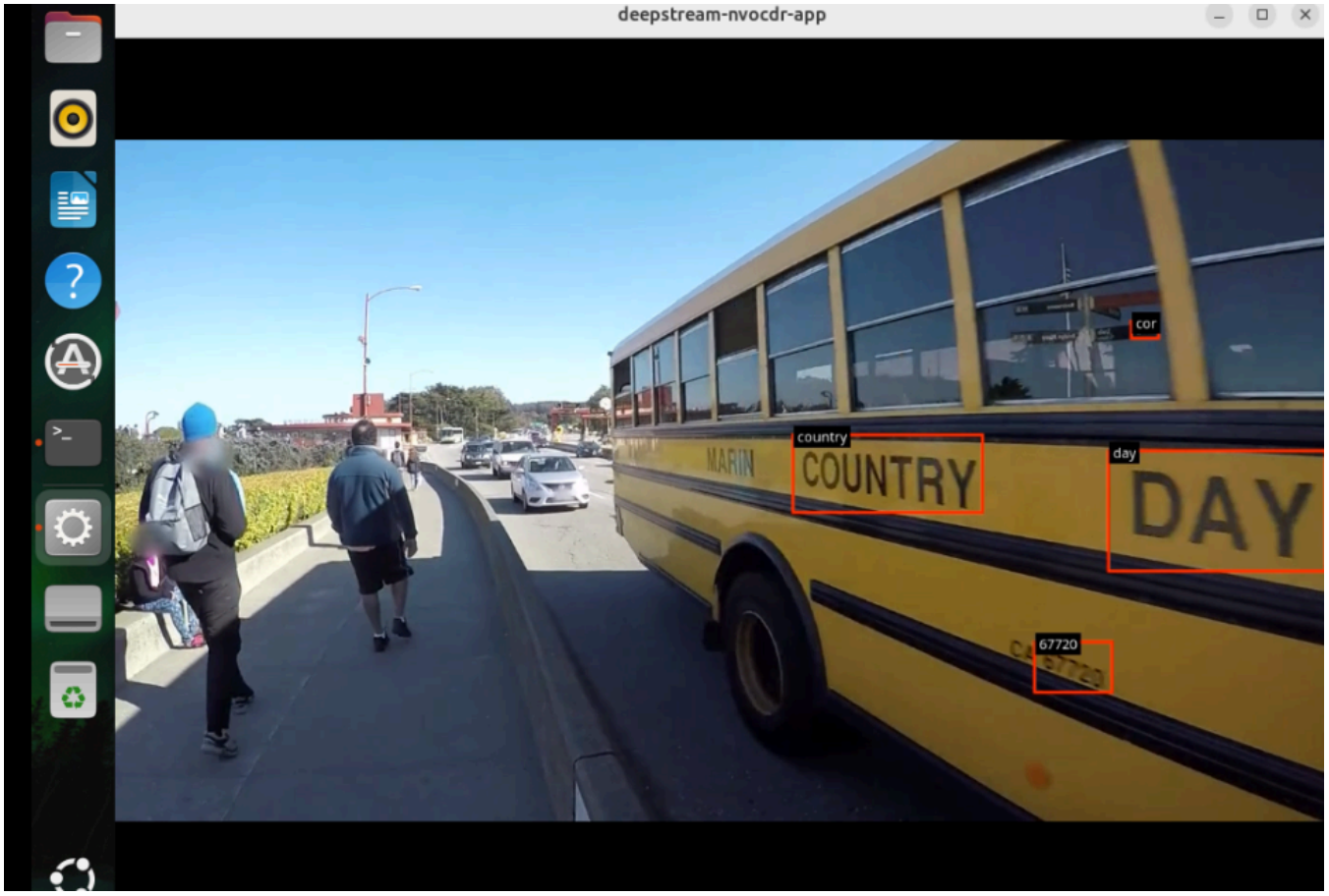
# 5. Running a local video

## 5.1 Run

```
cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_ocr.sh
```

The default video is:

```
/opt/nvidia/deepstream/deepstream-9.1/samples/streams/sample_720p.h264
```




The real-time preview window shows the detection boxes + the recognized text, at about 15 FPS.

## 5.2 Replace the video

```
./run_ocr.sh /home/jetson/Videos/test.mp4
```

The path must be an absolute path that exists on the Jetson; MP4/H.264 material is recommended.

## 5.3 Output location

In the video mode, the recognition results are overlaid on the real-time window as OSD. nvocdr does not generate an MP4 file by default (unless `output.type: 1` is configured in the YAML).

# 6. Real-time recognition with a USB camera

## 6.1 Run

```
cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_ocr_usb.sh
```




The script automatically detects the USB camera node through `~/find_usb_cam.sh`, so there is no need to specify it manually.

The default parameters fit the common USB UVC cameras:

```
Device: /dev/video0
Format: YUYV
Resolution: 640×480
Frame rate: 15 FPS
```

## 6.2 Camera pre-check

```
v4l2-ctl -d /dev/video0 --list-formats-ext
# Confirm the device number, the output format and whether it is occupied
```

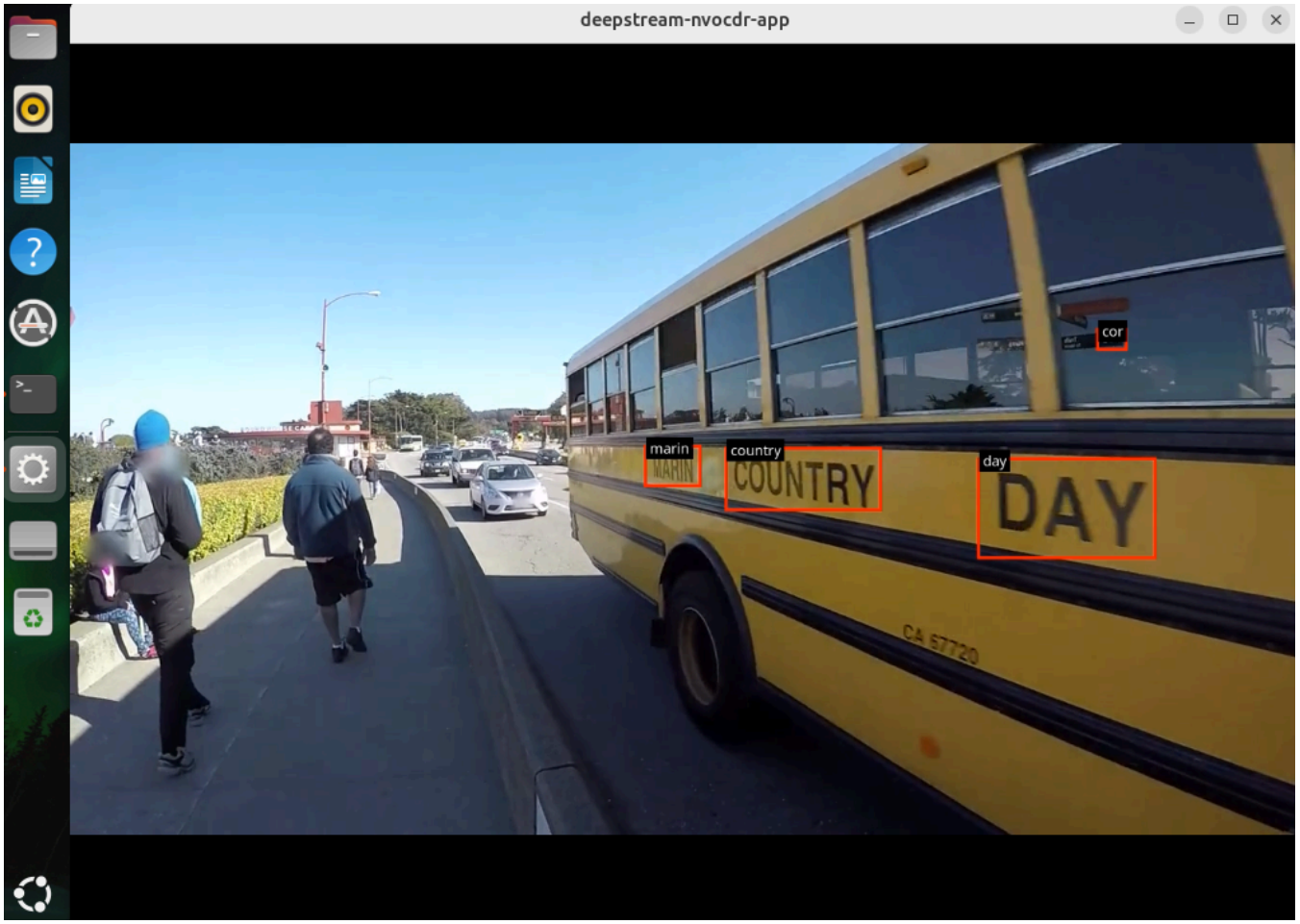
If the camera only supports MJPEG, you must first confirm the actual format, and then add a dedicated decoding adaptation for MJPEG.

# 7. Real-time recognition with a CSI camera

## 7.1 Run directly

Run:

```
cd ~/deepstream_tao_demo && export DISPLAY=:0 && ./run_ocr_csi.sh
```




The CSI entry point reads Argus directly through `csi://0` and no longer goes through `v4l2loopback`. The default parameters are `sensor-id=0`, 1920×1080, 30 FPS.

## 7.2 CSI no-picture check

If the terminal reports `No cameras available` or fails to open the device:

```
# Confirm that Argus can open the CSI camera directly
gst-launch-1.0 -v nvarguscamerasrc sensor-id=0 \
  ! 'video/x-raw(memory:NVMM),width=1920,height=1080,framerate=30/1,format=NV12' \
  ! fakesink

# Confirm the status of nvargus-daemon
systemctl status nvargus-daemon

# Confirm that the CSI camera is not occupied by another program
fuser -v /dev/video0
```

# 8. Notes on the current implementation

The `source-list` of nvocdr supports the following URI forms:

```
file:///path/to/video.mp4      # Local video file
rtsp://...                     # RTSP stream
http://...                     # HTTP stream
csi://0                        # CSI camera, sensor-id=0
```

**GStreamer pipeline strings are not supported**. The CSI camera uses the built-in `csi://<sensor-id>` source of the application:

```
CSI camera → nvarguscamerasrc → NVMM/NV12 → nvocdr
```

`run_ocr_csi.sh` writes `CSI_SENSOR_ID` into a temporary YAML and directly starts the Argus input.

# 9. Stopping and troubleshooting

The real-time window is displayed on the local Jetson desktop or the VNC desktop. To stop it, return to the terminal of the startup command and press `Ctrl+C` once.

| Symptom | Check first |
|---|---|
| Garbled text is recognized (letters and digits mixed together, forming no words) | The text itself is not within the 36-character set, or the input resolution is too low; this is a limitation of the OCR dictionary. |
| Chinese/uppercase text is not recognized | The current dictionary contains only 0-9 + a-z; use a dictionary that contains Chinese, or change the model. |
| No picture with USB | Use `v4l2-ctl -d /dev/video0 --list-formats-ext` to check the device number, the format and the occupancy. |
| No picture with CSI | Use `./run_ocr_csi.sh`; check `nvargus-daemon`, the CSI ribbon cable, the `sensor-id` and whether the camera is occupied. |
| `libnvocdr.so` fails to load | Confirm that `libnvocdr.so` and `libnvocdr_impl.so` exist under `/opt/nvidia/deepstream/deepstream-9.1/lib/`; `sudo ldconfig`. |
| The engine fails to load | Confirm that `~/deepstream_tao_apps/models/nvocdr/*.engine` exists and matches the current TensorRT version. |
| Only the window, no text | Check whether the text is clearly visible and the lighting is sufficient; check whether the OCRNet engine is loaded normally. |
| Very low FPS (< 5) | The OCDNet + OCRNet dual-model pipeline is heavy, so real-time inference is limited by itself; the input resolution can be adjusted. |

# 10. Quick reference of the key file paths

```
# Models
~/deepstream_tao_apps/models/nvocdr/
├── ocdnet_dcn_resnet18.onnx
├── ocdnet.fp16.engine
```

```
├── ocrnet_resnet50.onnx
├── ocrnet_resnet50.fp16.engine
└── character_list

# Libraries
/opt/nvidia/deepstream/deepstream-9.1/lib/
├── libnvocdr.so
└── libnvocdr_impl.so

# Applications
~/deepstream_tao_apps/apps/tao_others/deepstream-nvocdr-app/
└── deepstream-nvocdr-app

# Configuration
~/deepstream_tao_apps/configs/app/
├── nvocdr_app_config.yml       # video version
├── nvocdr_usb.yaml             # USB version (generated dynamically at runtime)
└── nvocdr_csi.yaml             # CSI version

# Scripts
~/deepstream_tao_demo/
├── run_ocr.sh                  # video
├── run_ocr_usb.sh              # USB
└── run_ocr_csi.sh              # CSI
```

# 11. References

- The NVIDIA DeepStream nvocdr app: https://github.com/NVIDIA/DeepStream/tree/main/src/apps/tao_others/deepstream-nvocdr-app
- The NVIDIA DeepStream SDK: https://docs.nvidia.com/metropolis/deepstream/dev-guide/index.html